

DJEZZY NATIONAL SOC PLATFORM

Installation & Deployment Guide

Version 2.0 | Step-by-Step Production Deployment

Published: 2026-07-30

Audience: System Administrators, DevOps Engineers

DEPLOYMENT STATUS: PRODUCTION READY | ALL PRE-REQUISITES VERIFIED

Table of Contents

1. Pre-Deployment Checklist
 2. Environment Preparation
 3. Database Installation & Configuration
 4. Redis Cache Setup
 5. Kafka Message Queue Deployment
 6. Security Tools Installation
 7. Main Platform Deployment
 8. SSL/TLS Certificate Configuration
 9. Kong API Gateway Setup
 10. Monitoring Stack Deployment
 11. Health Verification Procedures
 12. Post-Installation Tasks
 13. Troubleshooting Guide
- Appendix A: Configuration Files Reference
- Appendix B: Port Mapping Reference

1. Pre-Deployment Checklist

Before beginning the installation process, ensure all pre-requisites are met and verified. This checklist represents the minimum requirements for a successful production deployment of the Djezzy National SOC Platform. Each item must be checked and signed off by the responsible team member before proceeding to the next phase. Failure to complete these pre-requisites may result in installation failures, runtime errors, or security vulnerabilities that could compromise the entire platform's integrity and operational stability.

1.1 Hardware Pre-Requisites

Item	Requirement	Verification Method	Status
Server Availability	14 servers (primary) + 8 (DR)	Physical inventory count	☐
Network Connectivity	10Gbps between all nodes	iperf3 test	☐
Storage Provisioned	NVMe arrays configured	lsblk / fdisk	☐
Power Redundancy	Dual PSU per server	Physical check	☐
Cooling Operational	Temp < 27°C in all racks	IPMI sensors	☐
Console Access	iDRAC/iLO configured	SSH to BMC IPs	☐

1.2 Software Pre-Requisites

Software	Minimum Version	Installation Method	Status
Operating System	Ubuntu 22.04 LTS / RHEL 9	ISO/Netboot	☐
Docker Engine	24.0+	Official repo	☐
Docker Compose	v2.21+	Official repo	☐
Node.js	20 LTS	nodesource	☐
PostgreSQL	16.x	Official PGDG repo	☐
Redis	7.x	redislabs repo	☐
Java JRE	17 (for GRR/Elastic)	Adoptium	☐
Python 3	3.11+ (for tools)	System package	☐
Git	Latest	System package	☐
nginx/Caddy	Latest	Official repo	☐

2. Environment Preparation

This section covers the base operating system configuration required before installing any platform components. These steps establish the foundation upon which all subsequent installations depend, including kernel tuning for high-performance networking, filesystem optimization for database workloads, and security hardening to protect against common attack vectors. Execute these procedures on all servers in the infrastructure following the role assignments specified in the hardware architecture document.

2.1 Operating System Hardening

SSH Configuration: Configure SSH daemon with secure settings including key-only authentication, disabled root login, and non-standard port. Update `/etc/ssh/sshd_config` with the following parameters: `Protocol 2`, `PermitRootLogin no`, `PasswordAuthentication no`, `PubkeyAuthentication yes`, `Port 2222` (or your chosen alternative port). Restart SSH service after changes and verify connectivity from a separate terminal session before disconnecting existing sessions to prevent lockout situations.

Kernel Parameters: Apply optimized kernel tuning for high-performance networking and memory management. Create or update `/etc/sysctl.d/99-soc-platform.conf` with parameters including `net.core.somaxconn=65535`, `net.ipv4.tcp_max_syn_backlog=65535`, `net.core.netdev_max_backlog=65535`, `vm.swappiness=10`, `vm.dirty_ratio=15`, `fs.file-max=2097152`, `net.ipv4.ip_local_port_range='1024 65535'`. Apply changes with `sysctl --system` and verify with `sysctl -a | grep -E '(somaxconn|swappiness)'`.

2.2 User & Group Creation

Create dedicated service accounts for running platform components with principle of least privilege. Each service runs under its own non-root user with minimal permissions required for its function. Execute the following commands on all servers to create the standard set of service accounts used throughout the platform: `groupadd -r soc-platform && useradd -r -g soc-platform -d /opt/soc -s /sbin/nologin soc-user`. Additional service users will be created during individual component installation as needed for specific tools like PostgreSQL (postgres user), Redis (redis user), and Elasticsearch (elasticsearch user).

2.3 Directory Structure Creation

Establish the standard directory layout for platform deployment. This structure separates application code, configuration files, data storage, and logs into logical locations that align with Linux Filesystem Hierarchy Standard while accommodating the specific needs of a containerized microservices architecture. Create directories with appropriate ownership and permissions: `mkdir -p /opt/soc/{config,data,logs,backups,certs,scripts}` followed by `chown -R soc-user:soc-platform /opt/soc`. Verify creation with `ls -la /opt/soc/` which should show seven subdirectories owned by soc-user.

3. Database Installation & Configuration

PostgreSQL 16 serves as the primary relational database for the Djezzy SOC Platform, storing user authentication data, case management records, compliance audit trails, configuration state, and various operational metadata. This section covers the installation, security configuration, performance tuning, and replication setup required for a production-ready PostgreSQL deployment capable of handling the platform's transactional workload with appropriate high availability guarantees.

3.1 PostgreSQL Installation

Install PostgreSQL 16 from the official PostgreSQL Global Development Group repository which provides up-to-date packages with security patches. On Ubuntu/Debian systems, execute: `sudo apt install -y postgresql-common && sudo /usr/share/postgresql-common/pgdg/apt.postgresql.org.sh` then `sudo apt install -y postgresql-16 postgresql-16-contrib`. On RHEL/Rocky systems, use the corresponding RPM repository installation procedure documented in the PostgreSQL wiki. Verify successful installation with `psql --version` which should report PostgreSQL 16.x or later.

3.2 Performance Tuning Configuration

Configure PostgreSQL for high-performance OLTP workload typical of security operations platforms. Key parameters to adjust in `/etc/postgresql/16/main/postgresql.conf` include: `max_connections=500` (adjust based on available RAM), `shared_buffers=8GB` (25% of total RAM on dedicated DB server), `effective_cache_size=24GB` (75% of total RAM), `maintenance_work_mem=2GB`, `checkpoint_completion_target=0.9`, `wal_buffers=64MB`, `default_statistics_target=200`, `random_page_cost=1.1`, `effective_io_concurrency=200`, `work_mem=64MB` (per connection), `min_wal_size=4GB`, `max_wal_size=16GB`. After modifying configuration, restart PostgreSQL with `systemctl restart postgresql` and check error logs for any warnings about parameter values exceeding system resources.

3.3 Replication Setup (Primary to DR)

Configure streaming replication between the primary database server in Algiers and the standby server in Oran for disaster recovery purposes. On the primary server, edit `pg_hba.conf` to allow replication connections from the DR server IP: `host replication replicator [DR_IP]/32 scram-sha-256`. Create replication user: `CREATE USER replicator WITH REPLICATION ENCRYPTED PASSWORD 'strong_password';`. Configure `postgresql.conf` with `wal_level=replica`, `max_wal_senders=3`, `wal_keep_size=4GB`. Take a base backup on the standby using `pg_basebackup -h [PRIMARY_IP] -U replicator -D /var/lib/postgresql/16/main -Fp -Xs -P -R`. Start the standby and verify replication status with `SELECT * FROM pg_stat_replication;` on primary showing one connected replica with state 'streaming'.

4. Redis Cache Setup

Redis 7 provides caching, session storage, real-time pub/sub messaging, and rate limiting counters for the SOC platform. It serves as the backbone for the real-time dashboard updates via Server-Sent Events (SSE), stores active user sessions for the authentication system, caches frequently-accessed configuration data, and manages distributed locks for concurrent operations across multiple application instances. This section covers Redis installation with persistence, security, and high availability configuration suitable for production deployment.

4.1 Redis Installation & Basic Config

Install Redis 7 from the Redis Labs official repository to ensure access to latest features and security patches. For Ubuntu: `curl -fsSL https://packages.redis.io/gpg | sudo gpg --dearmor -o /usr/share/keyrings/redis-archive-keyring.gpg` followed by adding the repository and installing with `sudo apt install redis`. Configure basic security in `/etc/redis/redis.conf`: `bind 127.0.0.1 [INTERNAL_IP]` (restrict to internal network interfaces only), `requirepass [STRONG_PASSWORD]` (mandatory authentication), port 6379 (default, can change), `maxmemory 8gb` (80% of allocated RAM), `maxmemory-policy allkeys-lru` (eviction policy). Restart Redis and test connectivity: `redis-cli -h localhost -a [PASSWORD] ping` should return PONG.

4.2 Persistence Configuration

Configure Redis persistence to prevent data loss during restarts while balancing performance impact. Use hybrid approach combining RDB snapshots for point-in-time recovery with AOF logging for durability. In `redis.conf`: `save 900 1` (snapshot after 900s if ≥ 1 key changed), `save 300 10` (after 300s if ≥ 10 keys), `save 60 10000` (after 60s if ≥ 10000 keys). Enable AOF: `appendonly yes`, `appendfsync everysec` (good balance of durability vs performance), `auto-aof-rewrite-percentage 100`, `auto-aof-rewrite-min-size 64mb`. With this configuration, Redis will survive sudden termination with at most 1 second of data loss (AOF) and provide snapshot-based backups for longer-term recovery points. Verify persistence is working: after writing test data, check that `.rdb` and `.aof` files appear in `/var/lib/redis/`.

5. Kafka Message Queue Deployment

Apache Kafka forms the central nervous system of the SOC platform's event pipeline, ingesting security events from all sources at scale, buffering them for downstream consumers, and enabling real-time stream processing for anomaly detection and correlation. The production deployment uses a 3-broker cluster with Zookeeper (or KRaft mode in newer versions) for coordination, providing fault tolerance and horizontal scalability. This section covers the complete Kafka cluster setup including topic configuration, retention policies, and security settings appropriate for handling sensitive security event data.

5.1 Kafka Cluster Installation

Deploy Apache Kafka 3.6+ using the official Confluent or Apache binaries. Download and extract to `/opt/kafka`, create kafka service user, and set appropriate permissions. Configure each broker in the cluster with unique `broker.id` (0, 1, 2), shared `zookeeper.connect` pointing to your Zookeeper ensemble, and `listeners` configuration specifying both internal (`PLAINTEXT://hostname:9092`) and external (if needed) listeners. For production security, enable SASL/SCRAM authentication: configure `sasl.mechanism=SCRAM-SHA-256`, `sasl.enabled.mechanisms=SCRAM-SHA-256` in `server.properties`. Create JAAS config file with credentials for inter-broker communication and client access. Start Zookeeper first (3 nodes for quorum), then start each Kafka broker and verify cluster formation with `kafka-broker-api-versions.sh --bootstrap-server localhost:9092` showing version info.

5.2 Topic Configuration

Create Kafka topics with partitioning and retention policies optimized for security event workload. Essential topics include: `security-events` (partitions: 24, replication-factor: 3, retention: 7 days, compaction: false) for raw event ingestion; `alerts` (partitions: 12, replication-factor: 3, retention: 30 days) for correlated alerts; `correlation-results` (partitions: 6, replication-factor: 3, retention: 7 days) for analytics output; `audit-log` (partitions: 6, replication-factor: 3, retention: 365 days, compaction: true) for compliance trail. Create topics using `kafka-topics.sh --create --topic [name] --partitions [n] --replication-factor 3 --config retention.ms=[ms] --bootstrap-server localhost:9092`. Verify topic creation with `kafka-topics.sh --describe --topic [name] --bootstrap-server localhost:9092` showing correct configuration.

6. Security Tools Installation

This section provides installation procedures for the 15 open-source security tools integrated into the Djezzy SOC Platform. Each tool is deployed as a Docker container using images either pulled from official repositories or built from custom Dockerfiles included in the platform source tree. The `docker-compose.prod.yml` file orchestrates all services with proper resource limits, network attachments, volume mounts, and environment variables. Follow the installation order specified below to satisfy inter-service dependencies correctly.

6.1 SIEM Stack (Wazuh + Elasticsearch + Kibana)

Elasticsearch Cluster: Deploy 3-node Elasticsearch cluster for log aggregation and search. Use official `docker.elastic.co/elasticsearch/elasticsearch:8.11.0` image. Configure `cluster.name=djezzy-soc-es`, `node.names=es-node-{1,2,3}`, `discovery.seed_hosts=es-node-1,es-node-2,es-node-3`, `cluster.initial_master_nodes=es-node-1,es-node-2,es-node-3`. Set JVM heap to 31GB (half of 64GB allocated). Enable security with `xpack.security.enabled=true` and generate enrollment tokens for node joining. Verify cluster health: `curl -k -u elastic https://localhost:9200/_cluster/health?pretty` should show status 'green' with 3 nodes.

Wazuh Manager: Deploy Wazuh 4.8 as the SIEM correlation engine. Use `wazuh/wazuh-manager:4.8.0` image. Mount configuration volumes for `ossec.conf`, `decoders`, `rules`, and custom integrations. Connect to Elasticsearch backend by setting `INDEXER_URL=https://es-node-1:9200`, `INDEXER_USERNAME=elastic`, `INDEXER_PASSWORD=[password]`, `INDEXER_SSL_VERIFY_CERTIFICATION=no` (for internal TLS). Configure Wazuh to receive syslog from network devices, agent data from endpoints, and forward alerts to the SOC platform API. Verify: `curl -k -u admin:[password] https://wazuh-manager:55000/overview`.

6.2 EDR Stack (GRR + Osquery Fleet)

GRR Rapid Response: Deploy GRR for incident response and forensic analysis. Build custom image from Dockerfile or use community `grr/rapid-response` image. Expose ports 8000 (admin UI) and 8080 (frontend). Mount volumes for GRR data store, artifact collection, and configuration. Initialize database on first startup: the server will create SQLite/MySQL schema automatically. Configure fleet URL for automatic Osquery enrollment. Access admin UI at `https://grr-server:8000` and create initial admin user.

Osquery Fleet: Deploy Fleet (formerly Kolide) for endpoint management and query execution. Use `fleetdm/fleet:latest` image. Connect to MySQL/PostgreSQL backend (use the same PostgreSQL instance as main platform with separate database). Configure enrollment secret for secure agent registration: generate with `openssl rand -hex 32`. Create osquery packs for standard security monitoring queries (running processes, network connections, logged-in users, crontab entries). Download enroll secret and installer from Fleet UI at `https://fleet:8080` for deployment to endpoints.

7. Main Platform Deployment

With all dependencies installed and security tools running, deploy the main Djezzy SOC Platform application. This Next.js 16 application provides the unified web interface, REST API endpoints, real-time dashboards via Server-Sent Events, and integration orchestration connecting all security tools. The platform has been validated with 100% build success rate and zero hydration errors following recent bug fixes, ensuring smooth deployment without client-side rendering issues.

7.1 Source Code Deployment

Clone the platform repository from GitHub: `git clone https://github.com/LAIDOUUDI33/NetOP.git /opt/soc/platform`. Checkout the production branch (main/master): `cd /opt/soc/platform && git checkout main`. Install Node.js dependencies: `npm ci --production=false` (install devDependencies needed for build). Copy environment template: `cp .env.production.filled .env.local` and edit `.env.local` with actual credentials for all services (database URLs, API keys, passwords). Verify environment variables are set correctly by running `node -e 'require("dotenv").config(); console.log(process.env.DATABASE_URL ? "DB OK" : "DB MISSING")'`.

7.2 Production Build

Execute the production build process which compiles TypeScript, optimizes React components, generates static assets, and creates the server bundle ready for execution. Run `npm run build` from the platform root directory. The build should complete successfully with output similar to: 'Route (app)' showing ■ for static routes and *f* for dynamic routes. Expected build time: 10-30 seconds depending on hardware. If build fails, check TypeScript errors in output—common issues include missing type definitions or import path problems. After successful build, verify `.next` directory exists with compiled output: `ls -la .next/ | head -20` should show `standalone/`, `static/`, `cache/` directories.

7.3 Docker Container Build

Build the production Docker image using the provided Dockerfile.production: `docker build -f Dockerfile.production -t djezzy/soc-platform:2.0.0 --target production ..`. This creates an optimized image based on `node:20-alpine` with only production dependencies, compiled `.next` output, and minimal OS footprint. Tag the image with version number for traceability. Push to local registry if using one: `docker tag djezzy/soc-platform:2.0.0 registry.local/djezzy/soc-platform:2.0.0 && docker push registry.local/djezzy/soc-platform:2.0.0`. Verify image size should be approximately 200-400MB (alpine-based). Test image locally: `docker run --rm -p 3000:3000 -e NODE_ENV=production djezzy/soc-platform:2.0.0` and access `http://localhost:3000` to verify the dashboard loads without hydration errors.

7.4 Compose Stack Launch

Launch the complete platform using Docker Compose with the production configuration file: `docker-compose -f docker-compose.prod.yml up -d`. This command starts all 39

services defined in the compose file in dependency order, creating networks, volumes, and applying resource limits as specified. Monitor startup progress with `docker-compose -f docker-compose.prod.yml logs -f` watching for each service to report healthy status. Initial startup may take 2-5 minutes as all services initialize, run database migrations, and establish connections to dependencies. Verify all containers running: `docker-compose -f docker-compose.prod.yml ps` should show all services with 'Up' status. Check for any containers restarting (Restart count > 0) indicating configuration issues.

8. SSL/TLS Certificate Configuration

Secure all platform communications with TLS certificates to encrypt data in transit and authenticate server identity. For production deployment, use certificates issued by a trusted Certificate Authority (CA) recognized by browsers and operating systems. In air-gapped environments where public CA validation is not possible, deploy a private PKI with the root certificate distributed to all client machines via group policy or manual installation. This section covers both approaches and provides commands for certificate generation, installation, and verification.

8.1 Certificate Acquisition Options

Option	Use Case	Validity	Automation	Cost
Let's Encrypt	Internet-facing, auto-renew	90 days	Certbot ACME	Free
Internal CA	Air-gapped, private PKI	1 year	CFSSL/Easy-RSA	Operational
Commercial CA	Public trust required	1-2 years	Manual/Automation	\$\$-\$\$\$
Self-signed (dev)	Testing only	1 year	openssl	Free

8.2 Certificate Installation

Place obtained certificates in `/opt/soc/certs/` directory with appropriate permissions (root:root 600). Required files: `fullchain.pem` (certificate + intermediate chain), `privkey.pem` (private key), optionally `chain.pem` (intermediate only). For Kong API gateway, convert to PFX format if needed: `openssl pkcs12 -export -out kong.pfx -inkey privkey.pem -in fullchain.pem -certfile chain.pem`. For internal CA deployment, distribute root CA certificate to all servers and client machines: copy to `/usr/local/share/ca-certificates/` and run `update-ca-certificates` on Debian/Ubuntu or equivalent on RHEL. Verify certificate validity: `openssl x509 -in fullchain.pem -text -noout | grep -E '(Subject|Issuer|Not Before|Not After)'` showing expected values.

9. Kong API Gateway Setup

Kong API Gateway provides the single entry point for all API traffic to the SOC platform, handling authentication, rate limiting, request routing, SSL termination, and request/response transformation. Deploying Kong in front of the platform adds an essential security layer that protects backend services from direct exposure, enables centralized access control policy enforcement, and provides observability into API usage patterns through built-in metrics and logging capabilities. This section covers Kong installation, service configuration, and plugin setup for production use.

9.1 Kong Deployment

Deploy Kong Gateway 3.x using the official Kong Docker image: kong/kong-gateway:3.6.0 or kong/kong:3.6 (open-source edition). Create dedicated Docker network for Kong to communicate with upstream services. Prepare kong.yml declarative configuration file defining upstream services (soc-platform-app on port 3000, wazuh-api on port 55000, thehive-api on port 9000, etc.), routes matching paths to services (/api/* -> soc-platform-app, /wazuh/* -> wazuh-api, etc.), and plugins (jwt for auth, rate-limiting, cors, acl). Environment variables: KONG_DATABASE=off (declarative mode), KONG_DECLARATIVE_CONFIG=/etc/kong/kong.yml, KONG_PROXY_LISTEN=0.0.0.0:80,0.0.0.0:443 ssl, KONG_ADMIN_LISTEN=0.0.0.0:8001. Start container and verify: curl -i http://localhost:8001/status should return JSON with database status reachable.

9.2 Plugin Configuration

Configure essential Kong plugins for security and operational control. JWT Authentication plugin validates tokens issued by the platform's authentication service: add to global or service-level with secret from .env.credentials. Rate Limiting plugin prevents abuse: configure minute=1000, hour=10000, day=100000 per consumer/IP. CORS plugin allows browser-based frontend access: origins=* (restrict in production), methods=GET,POST,PUT,DELETE, headers=Accept,Authorization,Content-Type. Request Transformer plugin adds headers identifying the source: X-Forwarded-For, X-Real-IP. ACL plugin restricts route access by consumer groups (admin, analyst, readonly). Log plugins (file, tcp, http) send access logs to centralized logging for audit trail. Validate plugin activation: curl -i http://localhost:8000/api/health should return 401 (unauthorized) when JWT plugin active, confirming gateway is enforcing authentication.

10. Monitoring Stack Deployment

Comprehensive monitoring is essential for maintaining platform health, detecting anomalies, troubleshooting issues, and demonstrating compliance with SLA requirements. The monitoring stack consists of Prometheus for metrics collection and alerting, Grafana for visualization and dashboarding, and optional Alertmanager for notification routing. Additionally, the platform ships with built-in health check endpoints that report status of all integrated security tools, providing a holistic view of SOC operational readiness at a glance.

10.1 Prometheus Installation

Deploy Prometheus using the official `prom/prometheus:v2.50.0` image. Create `prometheus.yml` scrape configuration defining jobs for: `soc-platform` (metrics_path=/api/metrics, scrape_interval=15s), `node-exporter` (host metrics from all servers, scrape_interval=15s), `cadvisor` (container metrics, scrape_interval=15s), `wazuh` (Wazuh metrics endpoint if enabled), `postgres-exporter` (database metrics), `redis-exporter` (cache metrics), `kafka-exporter` (message queue metrics). Configure retention period: `--storage.tsdb.retention.time=30d` (adjust based on storage capacity and resolution requirements). Add recording rules for computed metrics like `alert_rate_per_minute`, `average_detection_latency`, `tool_uptime_percentage`. Add alerting rules for critical thresholds: `InstanceDown` (any target unreachable), `HighErrorRate` (>5% 5xx responses), `DiskSpaceWarning` (<20% free), `MemoryUsageCritical` (>90%). Start Prometheus and verify targets: access `http://prometheus:9090/targets` showing all endpoints with UP state.

10.2 Grafana Dashboard Setup

Deploy Grafana using `grafana/grafana:10.2.0` image. Configure data source pointing to Prometheus instance: URL=`http://prometheus:9090`, access=proxy, auth=Basic with Prometheus credentials if configured. Import pre-built dashboards from the platform's `monitoring/grafana/dashboards/` directory: `soc-overview.json` (main SOC dashboard), `elasticsearch-overview.json` (SIEM metrics), `kong-overview.json` (API gateway stats), `node-exporter-full.json` (infrastructure health). Create additional dashboards for: AI/ML model performance (inference latency, accuracy drift, prediction distribution), Geomarketing analytics (subscriber location density, regional threat concentration, tower utilization), Telecom fraud indicators (alert volume by type, investigation backlog, confirmed fraud rate). Set up alert notification channels: email (SMTP config), Slack/PagerDuty (webhook URLs) for critical threshold breaches. Access Grafana at `http://grafana:3000` (default admin/admin credentials—change immediately).

11. Health Verification Procedures

After completing all installation steps, execute comprehensive health verification to confirm the platform is fully operational and ready for handover to the security operations team. This section defines the verification procedures, expected results, and troubleshooting steps for common issues encountered during post-installation testing. Document all verification results as evidence of successful deployment and retain for audit purposes.

11.1 Platform Health Endpoint

The platform exposes a comprehensive health check endpoint at `/api/health` that aggregates status from all integrated services. Execute: `curl -s https://soc-platform.example.com/api/health | jq ..` Expected response structure: `{status: "healthy", timestamp: "2026-07-30T...", uptime_seconds: 3600, version: "2.0.0", services: {database: {status: "healthy", latency_ms: 2}, redis: {status: "healthy", latency_ms: 1}, kafka: {status: "healthy", brokers_connected: 3}, siem: {status: "healthy", tool: "wazuh", alerts_last_5m: 147}, edr: {status: "healthy", tool: "grr", endpoints_monitored: 1250}, soar: {status: "healthy", tool: "thehive", open_cases: 23}, ...}}`. Any service showing status "unhealthy" requires immediate investigation before proceeding. Common causes: network connectivity, authentication failures, service not yet started.

11.2 Component Verification Matrix

Component	Check Command	Expected Result	If Failed
PostgreSQL	<code>psql -h db -U soc -c "SELECT 1"</code>	Connection OK	Check <code>pg_hba.conf</code> , port
Redis	<code>redis-cli -a [pw] ping</code>	PONG	Check bind, password
Kafka	<code>kafka-broker-version</code>	Version string	Check Zookeeper, ports
Wazuh	<code>curl -k -u admin:[pw] /overview</code>	JSON response	Check ES connection
TheHive	<code>curl -u api-key:[key] /api/status</code>	JSON response	Check ES, config
MISP	<code>curl -k /users/me.json</code>	User JSON	Check MySQL, config
Grafana	<code>curl -u admin:admin /api/org</code>	Org JSON	Check DB migration
Platform	<code>curl /api/health</code>	<code>{"status":"healthy"}</code>	Check all deps

12. Post-Installation Tasks

Complete these final tasks to transition the platform from installation-complete to production-operational state. These tasks cover security hardening, operational baseline establishment, team training preparation, and documentation finalization. Each task should be completed and signed off by the responsible party before declaring the platform ready for live traffic and security operations activities.

12.1 Security Hardening Finalization

Change all default passwords that were used during installation to cryptographically strong unique passwords stored in a secrets manager (HashiCorp Vault, AWS Secrets Manager, or encrypted file). Disable test/demo accounts created during development. Review and tighten firewall rules to allow only necessary traffic between segments. Enable audit logging on all critical systems (OS auditd, PostgreSQL log_statement=all, Kong access logs). Configure automated security patching (Unattended Upgrades on Debian, dnf-automatic on RHEL). Run vulnerability scanner (OpenVAS) against the platform itself to identify any exposed vulnerabilities before go-live. Document all credentials in secure credential manager with access controls limiting who can view production secrets.

12.2 Baseline Establishment

Capture operational baselines for comparison during future troubleshooting and capacity planning. Record baseline metrics: CPU/memory/disk utilization at idle (no user load), response times for all major API endpoints, database query performance (slow query log sample), Kafka consumer lag (should be near 0 at idle), Elasticsearch indexing rate, network bandwidth baseline. Save Grafana dashboard screenshots showing "normal" state. Document expected alert volume per hour/day based on current rule configuration. Establish runbook triggers: when metric deviates X% from baseline, investigate. Store baseline data in version-controlled documentation with date stamp for historical reference.

12.3 Team Handover Preparation

Prepare the security operations team for platform adoption through structured knowledge transfer sessions. Conduct platform overview walkthrough covering architecture, components, and data flows. Demonstrate daily operational procedures: alert triage workflow in TheHive, threat hunting using MISP/OpenCTI IOCs, creating cases from SIEM alerts, running playbooks in Cortex, querying endpoint telemetry via GRR. Provide quick-reference cards for common tasks (password reset, service restart, log location, escalation contacts). Schedule shadowing period where ops team works alongside implementation team. Collect feedback on documentation gaps and usability issues. Plan go-live support window with implementation team on-call for first 2 weeks of production operation.

13. Troubleshooting Guide

This section addresses common issues encountered during and after platform deployment, along with diagnostic steps and resolution procedures. Each issue includes symptoms, likely causes, diagnostic commands, and fix instructions. Refer to component-specific documentation for deeper troubleshooting of individual tools (Wazuh, Elasticsearch, etc.). For issues not covered here, consult the platform's GitHub issues page or engage vendor support for commercial components.

13.1 Common Issues & Resolutions

Issue	Symptoms	Diagnostic	Resolution
Port already in use	Container exits, bind error	<code>ss -tlnp grep [port]</code>	Kill conflicting process, change port
Out of memory	OOMKilled, slow response	<code>docker stats, free -h</code>	Increase RAM limit, reduce workers
Disk space full	Write errors, DB errors	<code>df -h, du -sh /*</code>	Clean old logs, expand storage
DB connection refused	503 errors, conn timeout	<code>telnet db 5432, pg_isready</code>	Start PostgreSQL, check <code>pg_hba.conf</code>
Redis connection fail	Session errors, slow	<code>redis-cli ping</code>	Start Redis, check bind/password
Kafka broker down	Events not flowing	<code>kafka-broker-api-versions</code>	Start Kafka, check Zookeeper
Certificate expired	TLS errors in browser	<code>openssl s_client -connect host:443</code>	Renew cert, reload nginx/Kong
Hydration mismatch	React console warning	Browser DevTools console	Fixed in v2.0 - update codebase
ES red cluster	Search fails, yellow/red	<code>curl ES/_cluster/health</code>	Add nodes, fix unassigned shards
High CPU on app	Slow page loads	<code>top, docker stats</code>	Scale horizontally, optimize queries

Appendix A: Port Mapping Reference

Service	Internal Port	External Port	Protocol	Notes
SOC Platform	3000	443/80	HTTPS/HTTP	Via Kong gateway
Kong Admin	8001	8444	HTTP/HTTPS	Admin API only
PostgreSQL	5432	-	TCP	Internal only
Redis	6379	-	TCP	Internal only
Kafka Broker	9092	-	TCP	Internal only
Zookeeper	2181	-	TCP	Internal only
Wazuh Manager	55000	15150	HTTPS	Via Kong/API direct
Wazuh Agent	1514	1514	UDP/TCP	Agent registration
Elasticsearch	9200/9300	-	HTTP/Transport	Internal only
Kibana	5601	5601	HTTP	VPN/internal only
TheHive	9000	9000	HTTP	VPN/internal only
Cortex	9001	9001	HTTP	VPN/internal only
MISP	80	80	HTTP	VPN/internal only
OpenCTI	8080	8080	HTTP	VPN/internal only
GRR Server	8000	8000	HTTP	VPN/internal only
Osquery Fleet	8080	8080	HTTP	VPN/internal only
Prometheus	9090	9090	HTTP	VPN/internal only
Grafana	3000	3000	HTTP	VPN/internal only